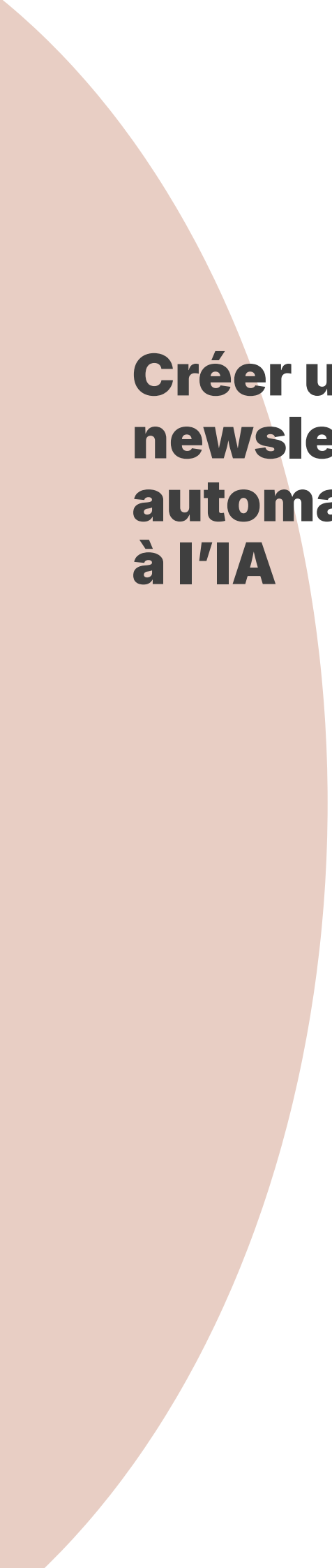




Créer une newsletter automatisée grâce à l'IA

Table des matières

I - Création de la newsletter automatisée grâce à l'IA	3
II - Aller plus loin et préparer une montée en charge	20
III - Synthèse	22
IV - Auto-évaluation	24
A. Exercice : Quiz	24
B. Exercice : Cas pratique.....	28
Solutions des exercices	29

I Création de la newsletter automatisée grâce à l'IA

Après avoir automatisé l'upsell, il est temps de relever un nouveau défi : soulager l'équipe marketing et fidéliser encore davantage la clientèle de Bon Vivant.

L'idée ici serait de permettre aux utilisateurs de recevoir une newsletter personnalisée, adaptée à leurs centres d'intérêt et à leur fréquence de réception préférée.

On pourrait imaginer la situation suivante.

Depuis le site web de Bon vivant, au lieu d'avoir simplement un champ pour entrer son adresse email, il pourrait y avoir une interface qui permettrait de sélectionner ses intérêts, ainsi que la fréquence à laquelle nous souhaitons recevoir des mails.

The mockup shows a sign-up form titled "Recevez votre newsletter Bon Vivant, vraiment à votre image." with the subtitle "Personnalisez votre expérience :". It features a section "Vos centres d'intérêts" with seven selectable options: Vin, Fromage, Jazz, Brunch, Voyages, Inspiration, and Promotions. Below this is a section "Que souhaitez-vous recevoir ?" with four checkboxes: "Idées de recettes", "Promotions exclusives", "Inspiration pour brunch", and "Suggestions de soirées". The "Fréquence" section has a dropdown menu set to "1x/semaine". The "Votre email" section contains a text input field with the placeholder "contact@test.com".

C'est ce que nous allons réaliser ensemble.

Pour cette V1, nous n'allons pas stocker le mail du client, ni l'envoyer.

À la place, nous allons tester le concept suivant : à partir de centres d'intérêts, est-il possible de rédiger un mail sans rédiger soi-même une seule ligne, grâce à l'IA ?

Contrairement aux fois précédentes nous allons tout de suite paramétrer notre frontend.

Rendez-vous dans votre dossier "frontend".

Depuis votre dossier frontend, rendez-vous dans le dossier "src" puis ouvrez le fichier "App.jsx".

```
1 cd frontend/src
```

Ajout de la route Newsletter

[Méthode](#)

Tout commence par l'ajout de la route Newsletter et nous allons également importer notre page.

```
<Route path="/newsletter" element={<NewsletterPage />} />

import NewsletterPage from "../pages/NewsletterPage";

1 import NewsletterPage from "../pages/NewsletterPage";
2 <Route path="/newsletter" element={<NewsletterPage />} />
```

Voici le rendu css final de la page.

```
frontend > src > App.jsx > App
1 import { BrowserRouter, Routes, Route } from "react-router-dom";
2 import ChatPage from "../pages/ChatPage";
3 import Navbar from "../components/Navbar";
4 import UpsellPage from "../pages/UpsellPage";
5 import Upsell from "../components/Upsell";
6 import NewsletterPage from "../pages/NewsletterPage";
7
8
9 export default function App() {
10   return (
11     <BrowserRouter>
12       <div className="min-h-screen bg-white">
13         { /* NAVBAR importée */ }
14         <Navbar />
15         <main className="flex justify-center items-start py-10">
16           <Routes>
17             <Route path="/" element={<ChatPage />} />
18             <Route path="/upsell" element={<UpsellPage />} />
19             <Route path="/newsletter" element={<NewsletterPage />} />
20           </Routes>
21         </main>
22       </div>
23     </BrowserRouter>
24   );
25 }
```

```
1 import { BrowserRouter, Routes, Route } from "react-router-dom";
2 import ChatPage from "../pages/ChatPage";
3 import Navbar from "../components/Navbar";
4 import UpsellPage from "../pages/UpsellPage";
5 import Upsell from "../components/Upsell";
6 import NewsletterPage from "../pages/NewsletterPage";
7
8
9 export default function App() {
10   return (
11     <BrowserRouter>
12     <div className="min-h-screen bg-white">
13       { /* NAVBAR importée */ }
14     <Navbar />
15     <main className="flex justify-center items-start py-10">
16     <Routes>
17     <Route path="/" element={<ChatPage />} />
18     <Route path="/upsell" element={<UpsellPage />} />
```

```
19 <Route path="/newsletter" element={<NewsletterPage />} />
20 </Routes>
21 </main>
22 </div>
23 </BrowserRouter>
24 );
25 }
```

Ensuite, nous allons nous rendre dans `pages/NewsletterPage` et créer le container de notre page. Nous allons également importer notre composant Newsletter.

```
demo@dev cd pages
```

```
1 cd pages
```

```
frontend > src > pages > NewsletterPage.jsx > ...
1 import NewsletterIA from "../components/NewsletterIA";
2
3 export default function NewsletterPage() {
4
5   return (
6
7     <div className="w-full max-w-xl">
8
9       <h1 className="text-3xl font-bold mb-8 text-center"> Démo Newsletter IA</h1>
10
11       <NewsletterIA />
12
13     </div>
14
15   );
16
17 }
```

```
1 import NewsletterIA from "../components/NewsletterIA";
2
3 export default function NewsletterPage() {
4
5   return (
6
7     <div className="w-full max-w-xl">
8
9       <h1 className="text-3xl font-bold mb-8 text-center"> Démo Newsletter IA</h1>
10
11       <NewsletterIA />
12
13     </div>
14
15   );
16
17 }
```

Puis enfin, nous allons créer le css ainsi que la logique associée. Pour ce faire, rendez-vous dans votre dossier `components`, puis dans le fichier `"NewsletterIA.jsx"`

```
demo@dev cd components
```

```
1 cd components
```

Dans le fichier `NewsletterIA.jsx`, nous allons inscrire notre logique.

```
1 import React, { useState } from "react";
2
3 const INTERESTS = [
4   { label: "Vin", icon: "🍷" },
5   { label: "Fromage", icon: "🧀" },
6   { label: "Jazz", icon: "🎷" },
7   { label: "Brunch", icon: "🍳" },
8   { label: "Voyages", icon: "🌍" },
9 ]
```

```

 9 { label: "Inspiration", icon: "💡" },
10 { label: "Promotions", icon: "🎁" }
11 ];
12
13 const WISHES = [
14 "Idées de recettes",
15 "Promotions exclusives",
16 "Inspiration pour brunch",
17 "Suggestions de soirées"
18 ];
19
20 const FREQUENCIES = [
21 "1x/semaine",
22 "1x/mois",
23 "À chaque nouveauté"
24 ];
25
26 export default function NewsletterIA() {
27   const [selectedInterests, setSelectedInterests] = useState([]);
28   const [selectedWishes, setSelectedWishes] = useState([]);
29   const [frequency, setFrequency] = useState(FREQUENCIES[0]);
30   const [email, setEmail] = useState("");
31   const [loading, setLoading] = useState(false);
32   const [newsletter, setNewsletter] = useState("");
33   const [error, setError] = useState("");
34
35   // Sélection/désélection d'un intérêt
36   const toggleInterest = (label) => {
37     setSelectedInterests((prev) =>
38       prev.includes(label) ? prev.filter((i) => i !== label) : [...prev, label]
39     );
40   };
41
42   // Sélection/désélection d'une envie
43   const toggleWish = (wish) => {
44     setSelectedWishes((prev) =>
45       prev.includes(wish) ? prev.filter((w) => w !== wish) : [...prev, wish]
46     );
47   };
48
49   // On envoie le formulaire
50   const handleSubmit = async (e) => {
51     e.preventDefault();
52     setError("");
53     setNewsletter("");
54     setLoading(true);
55     try {
56       // Pour la démo, on ne passe que les intérêts
57       const response = await fetch("http://localhost:8000/newsletter", {
58         method: "POST",
59         headers: { "Content-Type": "application/json" },
60         body: JSON.stringify({
61           interests: selectedInterests,
62           wishes: selectedWishes,
63           frequency,
64           email
65         }),
66       });
67       if (!response.ok) throw new Error("Erreur API");
68       const data = await response.json();
69       setNewsletter(data.newsletter || data.text || "Aucune newsletter générée.");
70     } catch (err) {
71       setError("Impossible de générer la newsletter.");

```

```

72 }
73 setLoading(false);
74 };
75
76 return (
77 <div className="max-w-xl mx-auto p-8 bg-white rounded-2xl shadow-2xl mt-12">
78 <h1 className="text-3xl font-semibold mb-1 text-center">Recevez votre newsletter
    <span className="font-bold text-green-700">Bon Vivant</span>, vraiment à votre image.
  </h1>
79 <p className="text-center mb-6 text-gray-500">Personnalisez votre expérience :</p>
80 <form onSubmit={handleSubmit} className="flex flex-col gap-5">
81 {/* Centres d'intérêts : grille visuelle */}
82 <div>
83 <label className="block mb-2 font-semibold">Vos centres d'intérêts</label>
84 <div className="grid grid-cols-4 gap-4">
85 {INTERESTS.map((item) => (
86 <button
87 type="button"
88 key={item.label}
89 className={`flex flex-col items-center justify-center rounded-xl border-2 h-20 text-
    2xl font-medium transition
90 ${selectedInterests.includes(item.el
91 ? "bg-green-100 border-green-500 text-green-900"
92 : "bg-gray-50 border-gray-200 text-gray-500 hover:border-green-300"}`}
93 onClick={() => toggleInterest(item.label)}
94 >
95 <span>{item.icon}</span>
96 <span className="text-xs mt-1">{item.label}</span>
97 </button>
98 )})
99 </div>
100 </div>
101
102 {/* Envies (checkbox group) */}
103 <div>
104 <label className="block mb-2 font-semibold">Que souhaitez-vous recevoir ?</label>
105 <div className="flex flex-wrap gap-4">
106 {WISHES.map((wish) => (
107 <label key={wish} className="flex items-center gap-1 text-gray-700">
108 <input
109 type="checkbox"
110 checked={selectedWishes.includes(wish)}
111 onChange={() => toggleWish(wish)}
112 className="accent-green-600"
113 />
114 <span>{wish}</span>
115 </label>
116 )})
117 </div>
118 </div>
119
120 {/* Fréquence */}
121 <div>
122 <label className="block mb-2 font-semibold">Fréquence</label>
123 <select
124 className="border rounded p-2 w-full"
125 value={frequency}
126 onChange={(e) => setFrequency(e.target.value)}
127 >
128 {FREQUENCIES.map((freq) => (
129 <option key={freq} value={freq}>{freq}</option>
130 )})
131 </select>

```



```

132 </div>
133
134 {/* Email */}
135 <div>
136 <label className="block mb-2 font-semibold">Votre email</label>
137 <input
138 type="email"
139 className="border rounded p-2 w-full"
140 placeholder="nom@email.com"
141 value={email}
142 onChange={(e) => setEmail(e.target.value)}
143 required
144 />
145 </div>
146
147 {/* Submit */}
148 <button
149 type="submit"
150 className="bg-green-600 text-white rounded-lg p-3 font-semibold text-lg shadow
    hover:bg-green-700 transition"
151 disabled={loading || !selectedInterests.length || !email}
152 >
153 {loading ? "Génération en cours..." : "Créer ma newsletter sur-mesure"}
154 </button>
155 </form>
156
157 {/* Preview Newsletter */}
158 {newsletter && (
159 <div className="mt-6 p-4 rounded-xl bg-blue-50 border border-blue-200 text-blue-900
    shadow-inner">
160 <div className="font-bold mb-1 text-lg">Aperçu de votre newsletter personnalisée :
    </div>
161 <div className="whitespace-pre-line">{newsletter}</div>
162 </div>
163 )}
164 {error && (
165 <div className="mt-4 p-3 rounded bg-red-50 border border-red-200 text-red-900">
166 {error}
167 </div>
168 )}
169 </div>
170 );
171 }

```

Ceci étant fait, nous pouvons passer au backend.

Comment créer une fonctionnalité newsletter ?

 Méthode

Depuis la racine de notre projet, nous allons créer notre fichier "newsletter_v1.0.txt"

```
demo@dev touch backend/core/prompts/newsletter_v1.0.txt
```

```
1 touch backend/core/prompts/newsletter_v1.0.txt
```

Dans ce fichier, nous allons cette fois ci créer un prompt en français.

```

backend > core > prompts > ✏ newsletter_v1.0.txt
1 Génère une newsletter hebdomadaire en Français, qui comporte 3 sections : actualités, conseils, promotions.
2 Le lecteur est intéressé par : {interests}.
3 Rédige des titres accrocheurs et des paragraphes courts.

```

- 1 Génère une newsletter hebdomadaire en Français, qui comporte 3 sections : actualités, conseils, promotions.
- 2 Le lecteur est intéressé par : {interests}.
- 3 Rédige des titres accrocheurs et des paragraphes courts.

Ensuite, dans le fichier "services.py", nous allons définir la fonction qui utilise ce prompt et qui génère également la newsletter.

```
1 cd ..
```

Nous n'avons pas besoin de définir à nouveau notre LLM car nous l'avons fait pour le précédent endpoint.

Nous commençons par charger notre prompt dans la variable PROMPT_NEWSLETTER.

```
PROMPT_NEWSLETTER = "newsletter_v1.0.txt"
```

```
1 PROMPT_NEWSLETTER = "newsletter_v1.0.txt"
```

Ensuite, nous allons créer notre mode démo.

```
USE_DEMO_NEWS = os.environ.get("USE_DEMO_NEWS", "0") == "1"
```

```
1 USE_DEMO_NEWS = os.environ.get("USE_DEMO_NEWS", "0") == "1"
```

Nous commençons donc par créer la fonction generate_newsletter.

En paramètre, on met interests, qui est une liste de chaîne de caractères (list[str]) qui représente les centres d'intérêt de l'utilisateur.

En retour, il y a un texte sous forme de chaîne de caractères.

```
def generate_newsletter(interests: list[str]) -> str:
```

Ensuite, nous allons charger notre prompt.

```
prompt_template = load_prompt(PROMPT_NEWSLETTER)
```

La prochaine étape consiste à insérer les intérêts de l'utilisateur.

```
# On remplace par la liste d'intérêts de l'utilisateur
prompt = prompt_template.format(interests=", ".join(interests))
```

Nous paramétrons un mode démo pour le testing. Celui-ci nous permettra de tester la logique sans utiliser l'IA. Si le mode démo est activé, on renvoie directement le texte ci-dessous.

```
if USE_DEMO_NEWS:
    return (
        "Actualités : Toutes les nouveautés sur " + ", ".join(interests) + ".\n"
        "Conseils : Découvrez nos astuces et nos conseils.\n"
        "Promotions : Offres spéciales réservées aux abonnés"
    )
```

Enfin, si le mode démo n'est pas activé, on appelle l'IA grâce à la fonction llm en y renseignant plusieurs paramètres.

Premièrement, prompt qui correspond au texte personnalisé.

Ensuite, max_tokens qui fait référence à la taille maximale de la réponse.

Result, est la réponse brute de l'IA

Enfin, on nettoie la réponse avec return result["choices"][0]["text"].strip()

```
result = llm(prompt, max_tokens=512)
return result["choices"][0]["text"].strip()
```

Le résultat final est celui-ci.

```
def generate_newsletter(interests: list[str]) -> str:
    # On charge le prompt
    prompt_template = load_prompt(PROMPT_NEWSLETTER)

    # On remplace par la liste d'intérêts de l'utilisateur
    prompt = prompt_template.format(interests=", ".join(interests))

    # Mode démo = renvoyer un faux texte sans appeler le modèle
    if USE_DEMO_NEWS:
        return (
            "Actualités : Toutes les nouveautés sur " + ", ".join(interests) + ".\n"
            "Conseils : Découvrez nos astuces et nos conseils.\n"
            "Promotions : Offres spéciales réservées aux abonnés"
        )

    # Sinon on appelle le LLM
    result = llm(prompt, max_tokens=512)
    return result["choices"][0]["text"].strip()
```

```
1 def generate_newsletter(interests: list[str]) -> str:
2     # On charge le prompt
3     prompt_template = load_prompt(PROMPT_NEWSLETTER)
4     # On remplace par la liste d'intérêts de l'utilisateur
5     prompt = prompt_template.format(interests=", ".join(interests))
6     # Mode démo = renvoyer un faux texte sans appeler le modèle
7     if USE_DEMO_NEWS:
8         return (
9             "Actualités : Toutes les nouveautés sur " + ", ".join(interests) + ".\n"
10            "Conseils : Découvrez nos astuces et nos conseils.\n"
11            "Promotions : Offres spéciales réservées aux abonnés"
12        )
13    # Sinon on appelle le LLM
14    result = llm(prompt, max_tokens=512)
15    return result["choices"][0]["text"].strip()
```

Et votre fichier services.py au complet devrait être similaire à ceci.

```

backend > core > # services.py > ...
1 import json
2 import os
3 import torch
4 from transformers import pipeline
5
6 from core.prompt_manager import load_prompt
7 from llama_cpp import Llama
8
9 PROMPT_UPSELL = 'upsell_prompt_v1.0.txt'
10 PROMPT_NEWSLETTER = 'newsletter_v1.0.txt'
11
12 MODEL_PATH = os.path.join(
13     os.path.dirname(__file__),
14     '..',
15     'models',
16     'mistral-7b-q4_k_m.gguf'
17 )
18
19 llm = Llama(model_path=MODEL_PATH, n_gpu_layers=16, n_threads=4)
20
21 USE_DEMO_TEXT = os.environ.get("USE_DEMO_TEXT", "0") == "1"
22 USE_DEMO_SENTIMENT = os.environ.get("USE_DEMO_SENTIMENT", "0") == "1"
23 _sentiment_pipeline = None # Variable pour lazy loading
24 USE_DEMO_NEWS = os.environ.get("USE_DEMO_NEWS", "0") == "1"
25
26 DATASET_PATH = os.path.join(os.path.dirname(__file__), "../data/products.json")
27
28 def load_products():
29     with open(DATASET_PATH, 'r', encoding='utf-8') as f:
30         return json.load(f)
31
32 PRODUCTS_DATA = load_products()
33
34 def find_product_by_name(query: str):
35     query = query.lower()
36     for product in PRODUCTS_DATA:
37         if product['name'].lower() in query:
38             return product
39     return None
40
41 def suggest_wine_for_product(product):
42     pairing = {
43         "Fromage": "Un vin blanc sec, type Sancerre ou Chablis.",
44         "Poisson": "Un vin blanc fruité, type Sauvignon ou Chardonnay.",
45         "Charcuterie": "Un vin rouge léger, type Pinot Noir.",
46         "Dessert": "Un vin moelleux, type Sauternes"
47     }
48     suggestion = pairing.get(product["category"], "Un vin adapté à ce plat, à voir selon vos goûts !")
49     return f"Pour {product['name']}, je te recommande {suggestion}"
50
51 def get_product_info(user_message: str):
52     if USE_DEMO_TEXT:
53         return "Mode DEMO : Produit détecté -> suggestion de vin fictive."
54     product = find_product_by_name(user_message)
55     if not product:
56         return None
57
58     message_lower = user_message.lower()
59     if any(x in message_lower for x in [
60         "que contient",
61         "description",
62         "qu'y a-t-il dans",
63         "c'est quoi",
64         "qu'est-ce que"
65     ]):
66         return f"{product['name']} : {product['description']}"
67     else:
68         return suggest_wine_for_product(product)
69
70 # Analyse de sentiment
71 def get_sentiment_pipeline():
72     """
73     Charge le pipeline HuggingFace pour l'analyse de sentiment.
74     Utilise PyTorch en backend.
75     Chargement unique (singleton).
76     """
77
78     global _sentiment_pipeline
79     if _sentiment_pipeline is None:
80         _sentiment_pipeline = pipeline("sentiment-analysis")
81     return _sentiment_pipeline
82
83 def analyze_sentiment(text):
84
85     if USE_DEMO_SENTIMENT:
86         return "neutral", 0.5
87
88     pipe = get_sentiment_pipeline()
89     result = pipe(text)
90     label = result[0]["label"].lower()
91     score = result[0]["score"]
92
93     return label, score
94
95 # ----- Fonction upsell
96
97 def get_upsell(cart_items: list[str]) -> dict:
98     prompt_template = load_prompt(PROMPT_UPSELL)
99     prompt = prompt_template.replace("{cart_items}", ", ".join(cart_items))
100     result = llm(prompt, max_tokens=50)
101     return {"suggestion": result["choices"][0]["text"].strip()}
102
103 # ----- Fonction newsletter
104
105 def generate_newsletter(interests: list[str]) -> str:
106     # On charge le prompt
107     prompt_template = load_prompt(PROMPT_NEWSLETTER)
108
109     # On remplace par la liste d'intérêts de l'utilisateur
110     prompt = prompt_template.format(interests=", ".join(interests))
111
112     # Mode démo = renvoyer un faux texte sans appeler le modèle
113     if USE_DEMO_NEWS:
114         return [
115             "Actualités : Toutes les nouveautés sur " + ", ".join(interests) + ".\n"
116             "Conseils : Découvrez nos astuces et nos conseils.\n"
117             "Promotions : Offres spéciales réservées aux abonnés"
118         ]
119
120     # Sinon on appelle le LLM
121     result = llm(prompt, max_tokens=512)
122     return result["choices"][0]["text"].strip()
123
124

```

```

1 import json
2 import os
3 import torch
4 from transformers import pipeline
5
6 from core.prompt_manager import load_prompt
7 from llama_cpp import Llama
8

```

```

9 PROMPT_UPSELL = 'upsell_prompt_v1.0.txt'
10 PROMPT_NEWSLETTER = "newsletter_v1.0.txt"
11
12 MODEL_PATH = os.path.join(
13 os.path.dirname(__file__),
14 '..',
15 'models',
16 'mistral-7b-q4_k_m.gguf'
17 )
18
19 llm = Llama(model_path=MODEL_PATH, n_gpu_layers=16, n_threads=4)
20
21 USE_DEMO_TEXT = os.environ.get("USE_DEMO_TEXT", "0") == "1"
22 USE_DEMO_SENTIMENT = os.environ.get("USE_DEMO_SENTIMENT", "0") == "1"
23 _sentiment_pipeline = None # Variable pour lazy loading
24 USE_DEMO_NEWS = os.environ.get("USE_DEMO_NEWS", "0") == "1"
25
26 DATASET_PATH = os.path.join(os.path.dirname(__file__), "../data/products.json")
27
28 def load_products():
29 with open(DATASET_PATH, 'r', encoding='utf-8') as f:
30 return json.load(f)
31
32 PRODUCTS_DATA = load_products()
33
34 def find_product_by_name(query: str):
35 query = query.lower()
36 for product in PRODUCTS_DATA:
37 if product['name'].lower() in query:
38 return product
39 return None
40
41 def suggest_wine_for_product(product):
42 pairing = {
43 "Fromage": "Un vin blanc sec, type Sancerre ou Chablis.",
44 "Poisson": "Un vin blanc fruité, type Sauvignon ou Chardonnay.",
45 "Charcuterie": "Un vin rouge léger, type Pinot Noir.",
46 "Dessert": "Un vin moelleux, type Sauternes"
47 }
48 suggestion = pairing.get(product["category"], "Un vin adapté à ce plat, à voir selon
    vos goûts !")
49 return f"Pour {product['name']}, je te recommande {suggestion}"
50
51 def get_product_info(user_message: str):
52 if USE_DEMO_TEXT:
53 return "Mode DEMO : Produit détecté -> suggestion de vin fictive."
54 product = find_product_by_name(user_message)
55 if not product:
56 return None
57
58 message_lower = user_message.lower()
59 if any(x in message_lower for x in [
60 "que contient",
61 "description",
62 "qu'y a-t-il dans",
63 "c'est quoi",
64 "qu'est-ce que"
65 ]):
66 return f"{product['name']} : {product['description']}"
67 else:
68 return suggest_wine_for_product(product)
69
70 # Analyse de sentiment

```

```

71 def get_sentiment_pipeline():
72     """
73     Charge le pipeline HuggingFace pour l'analyse de sentiment.
74     Utilise PyTorch en backend.
75     Chargement unique (singleton).
76     """
77     global _sentiment_pipeline
78     if _sentiment_pipeline is None:
79         _sentiment_pipeline = pipeline("sentiment-analysis")
80     return _sentiment_pipeline
81
82 def analyze_sentiment(text):
83     if USE_DEMO_SENTIMENT:
84         return "neutral", 0.5
85
86     pipe = get_sentiment_pipeline()
87     result = pipe(text)
88     label = result[0]["label"].lower()
89     score = result[0]["score"]
90
91     return label, score
92
93 # ---- Fonction upsell
94
95 def get_upsell(cart_items: list[str]) -> dict:
96     prompt_template = load_prompt(PROMPT_UPSELL)
97     prompt = prompt_template.replace("{cart_items}", ", ".join(cart_items))
98     result = llm(prompt, max_tokens=50)
99     return {"suggestion": result["choices"][0]["text"].strip()}
100
101 # ---- Fonction newsletter
102
103 def generate_newsletter(interests: list[str]) -> str:
104     # On charge le prompt
105     prompt_template = load_prompt(PROMPT_NEWSLETTER)
106     # On remplace par la liste d'intérêts de l'utilisateur
107     prompt = prompt_template.format(interests=", ".join(interests))
108     # Mode démo = renvoyer un faux texte sans appeler le modèle
109     if USE_DEMO_NEWS:
110         return (
111             "Actualités : Toutes les nouveautés sur " + ", ".join(interests) + ".\n"
112             "Conseils : Découvrez nos astuces et nos conseils.\n"
113             "Promotions : Offres spéciales réservées aux abonnés"
114         )
115     # Sinon on appelle le LLM
116     result = llm(prompt, max_tokens=512)
117     return result["choices"][0]["text"].strip()

```

Il est temps de passer à la création de nos classes Pydantic de requête/réponse.

Commencez par vous rendre dans le fichier schemas.py.

```
demo@dev cd ..
```

```
demo@dev cd api
```

```

1 cd ..
2 cd api

```

Puis nous allons créer nos requêtes à la suite des précédentes requêtes.

```
#Requête Newsletter
```

```
class NewsletterRequest(BaseModel):
```

```
    interests: list[str]
```

```
#Réponse Newsletter
```

```
class NewsletterResponse(BaseModel):
```

```
    newsletter: str
```

```
1 #Requête
2 class NewsletterRequest(BaseModel):
3     interests: list[str]
4 #Réponse
5 class NewsletterResponse(BaseModel):
6     newsletter: str
```

Au final, votre fichier schemas.py devrait être similaire à celui-ci.

```

backend > api > 🔄 schemas.py > ...
1  from pydantic import BaseModel
2  from typing import List, Optional
3
4
5  #Requête Chat
6
7  class ChatRequest(BaseModel):
8      session_id: Optional[str] = None
9      user_message: str
10     products: Optional[List[str]] = []
11
12     # Réponse Chat
13
14
15     class ChatResponse(BaseModel):
16         recommendation: str
17         sentiment: Optional[str] = None
18         escalate_to_human: bool = False
19
20     // # On définit les schémas de requête Upsell
21     class UpsellRequest(BaseModel):
22         cart_items: List[str]
23
24     // # On définit les schémas de réponse Upsell
25     class UpsellResponse(BaseModel):
26         suggestion: str
27
28     #Requête Newsletter
29
30     class NewsletterRequest(BaseModel):
31
32         interests: list[str]
33
34     #Réponse Newsletter
35
36     class NewsletterResponse(BaseModel):
37
38         newsletter: str
39

```

```

1 from pydantic import BaseModel
2 from typing import List, Optional
3
4
5 #Requête Chat
6
7 class ChatRequest(BaseModel):
8 session_id: Optional[str] = None

```



```

9 user_message: str
10 products: Optional[List[str]] = []
11
12 # Réponse Chat
13
14
15 class ChatResponse(BaseModel):
16     recommendation: str
17     sentiment: Optional[str] = None
18     escalate_to_human: bool = False
19
20 # On définit les schémas de requête Upsell
21 class UpsellRequest(BaseModel):
22     cart_items: List[str]
23
24 # On définit les schémas de réponse Upsell
25 class UpsellResponse(BaseModel):
26     suggestion: str
27
28 #Requête Newsletter
29
30 class NewsletterRequest(BaseModel):
31
32     interests: list[str]
33
34 #Réponse Newsletter
35
36 class NewsletterResponse(BaseModel):
37
38     newsletter: str

```

Une fois terminé, nous allons créer notre endpoint newsletter dans le fichier main.py

Création de l'endpoint newsletter

 Méthode

1. Nous commençons par l'import de nos classes Pydantic.

```
from api.schemas import ChatRequest, ChatResponse, UpsellRequest, UpsellResponse, NewsletterRequest, NewsletterResponse
```

```
1 from api.schemas import ChatRequest, ChatResponse, UpsellRequest,
    UpsellResponse, NewsletterRequest, NewsletterResponse
```

Ensuite, nous allons importer la fonction `generate_newsletter` que l'on a créée précédemment. Celle-ci nous permettra de générer le texte de la newsletter à partir d'intérêts donnés par l'utilisateur.

```
from core.services import get_upsell, generate_newsletter
```

```
1 from core.services import get_upsell, generate_newsletter
```

2. Maintenant, nous allons définir l'endpoint.

Tout commence par le décorateur `@app.post(...)` : ce décorateur FastAPI permet de créer un endpoint HTTP de type POST. Il sera accessible à l'URL `/newsletter`

Ensuite, grâce à `response_model=NewsletterResponse`, FastAPI sait que la réponse doit respecter le schéma `NewsletterResponse` qui sera automatiquement validé par Pydantic.

```
@app.post("/newsletter", response_model=NewsletterResponse)
```

```
1 @app.post("/newsletter", response_model=NewsletterResponse)
```

3. Ensuite nous allons créer la fonction de l'endpoint.

Cette fonction sera asynchrone grâce à « `async` » afin de pouvoir gérer plusieurs requêtes en même temps.

Celle-ci attend un paramètre req, de type NewsletterRequest car il s'agit du modèle Pydantic que nous avons défini.

```
async def newsletter_endpoint(req: NewsletterRequest):
```

```
1 async def newsletter_endpoint(req: NewsletterRequest):
```

4. Maintenant, nous allons appeler la logique métier.

Pour ce faire, nous allons appeler la fonction generate_newsletter en passant req.interests (cela fait référence aux centres d'intérêt envoyés par l'utilisateur)

Text contient le contenu généré de la newsletter en chaine de caractères.

```
text = generate_newsletter(req.interests)
```

```
1 text = generate_newsletter(req.interests)
```

5. Enfin, nous allons retourner la réponse.

On renvoie un objet NewsletterResponse. Ce modèle Pydantic définit la structure de sortie et FastAPI convertit automatiquement en JSON pour la réponse HTTP.

```
return NewsletterResponse(newsletter=text)
```

```
1 return NewsletterResponse(newsletter=text)
```

Le bloc de code final devrait être similaire à ceci.

```
1 @app.post("/newsletter", response_model=NewsletterResponse)
2 async def newsletter_endpoint(req: NewsletterRequest):
3     text = generate_newsletter(req.interests)
4     return NewsletterResponse(newsletter=text)
```

Il est maintenant temps de tester notre route.

Lancez votre serveur backend ainsi que votre serveur frontend et testez depuis la fonction newsletter depuis le frontend.

Commençons par le backend.

```
demo@dev cd backend
```

```
demo@dev source .venv/bin/activate
```

```
demo@dev uvicorn api.main:app --reload
```

```
1 cd backend
2 source .venv/bin/activate
3 uvicorn api.main:app --reload
```

Puis nous allons passer au frontend.

```
demo@dev cd frontend
```

```
demo@dev npm run dev
```

```
1 cd frontend
2 npm run dev
```

1x/semaine

Votre email

Créer ma newsletter sur-mesure

Aperçu de votre newsletter personnalisée :

Title 1: "Fromage de la semaine : Le Reblochon, un fromage savoyard à goût intemporel"
Paragraphe: "Découvrez notre fromage de la semaine, le Reblochon, un fromage savoyard à saveur riche et intemporel. Confectionné à base de cru de vache, ce fromage a une texture croquante et un goût à la saveur de crème."

Title 2: "Promotion hebdomadaire : Enregistrez des économies avec nos fromages de la semaine"
Paragraphe: "Enregistrez des économies avec notre promotion hebdomadaire sur nos fromages de la semaine! Achetés seuls ou en lots, vous bénéficierez d'un rabais spécial. Économisez sur des délices tels que le Reblochon, le Comté ou le Roquefort."

Title 3: "Conseils de l'expert : Comment sélectionner les meilleurs fromages pour vous?"
Paragraphe: "Découvrez les conseils de notre expert pour

II Aller plus loin et préparer une montée en charge

Aujourd'hui, notre solution fonctionne en local. C'est idéal pour prototyper, mais limité car seule Emilie ou un utilisateur sur la même machine peut accéder à l'outil. Demain nous pourrions déployer la solution sur un serveur cloud tels que OVH ou AWS combiné à Docker.

OVHcloud, AWS et Docker

Az Définition

OVHcloud (<https://www.ovhcloud.com/fr/>) est un service européen basé en France. Il respecte les normes européennes en matière de protection de données (RGPD). C'est l'idéal si on veut privilégier la souveraineté des données et limiter la dépendance aux services américains. C'est une des raisons qui explique qu'OVH est souvent choisi par des entreprises qui souhaitent rester dans un cadre réglementaire européen strict.

Quant à lui, AWS (Amazon Web Services : <https://aws.amazon.com/fr/>) est le leader mondial du cloud. Il est reconnu pour sa fiabilité et sa flexibilité. AWS propose une très grande variété de services. C'est une solution qui permet de démarrer rapidement, d'augmenter la puissance de calcul à la demande et de connecter facilement d'autres services cloud.

Enfin, Docker (<https://www.docker.com/>) permet de déployer notre application dans un conteneur, quelle que soit la le service cloud choisi. L'avantage ici va être de garantir la portabilité de l'application. C'est-à-dire que l'on peut déplacer l'application d'un serveur à un autre sans réécrire le code.

Cela permettrait d'ouvrir l'accès à tous les utilisateurs du site simultanément.

Autre évolution potentielle, le stockage des données.

Aujourd'hui toutes les données sont conservées en local.

Dans une approche plus scalable il serait intéressant de migrer ces données vers une base de données centralisée ou un stockage cloud tels que PostgreSQL <https://www.postgresql.org/> ou encore Mongo DB (<https://www.mongodb.com/>)

PostgreSQL et Mongo DB

Az Définition

PostgreSQL est une base de données relationnelle de type SQL fiable et très robuste. C'est l'idéal lorsqu'on a des données structurées comme des fiches clients par exemple ou tout ce qui nécessite des relations claires entre les données.

Dans le cadre de Bon Vivant, PostgreSQL permettrait de facilement associer chaque commande à un client et donc accéder à des informations client comme "montre-moi toutes les newsletters générées par {client} au cours du mois d'août"

Quant à elle, MongoDB est une base de données dites non relationnelle (noSQL). Les données sont stockées en documents JSON. L'avantage de MongoDB est qu'il y a une grande souplesse. Chaque enregistrement peut avoir une structure qui lui est propre. MongoDB est également très performant lorsqu'il s'agit de stocker et récupérer des données non structurées. Enfin, dernier avantage, mongoDB s'adapte bien aux évolutions. Nous n'avons pas besoin de redéfinir un schéma complexe si la structure change.

Dans le cadre de Bon Vivant, MongoDB pourrait être utilisée pour stocker le contenu brut de chaque newsletter générée. Chaque document JSON pouvant être différent selon les intérêts ou les besoins de l'utilisateur.

Grâce à ça, plusieurs utilisateurs pourraient accéder aux mêmes informations sans dépendre d'une seule machine.

Enfin, pour garantir la performance de notre solution, on pourrait aussi optimiser l'IA.

Nous pourrions héberger le modèle sur une machine plus puissante, passer par un GPU cloud ou encore passer par une API externe telle que OpenAI.

Cela permettrait de bénéficier d'une puissance de calcul plus importante et donc d'avoir un texte encore plus qualitatif.

III Synthèse

A. Flashcards

Newsletter automatisée IA

Envoi personnalisé de newsletters générées automatiquement par intelligence artificielle.

Prompt IA versionné

Fichier texte contenant les instructions IA, stocké pour suivi et modifications.

Mode démo

Fonctionnalité pour tester sans appel réel à l'intelligence artificielle.

PostgreSQL

Base de données relationnelle adaptée aux données structurées et relations.

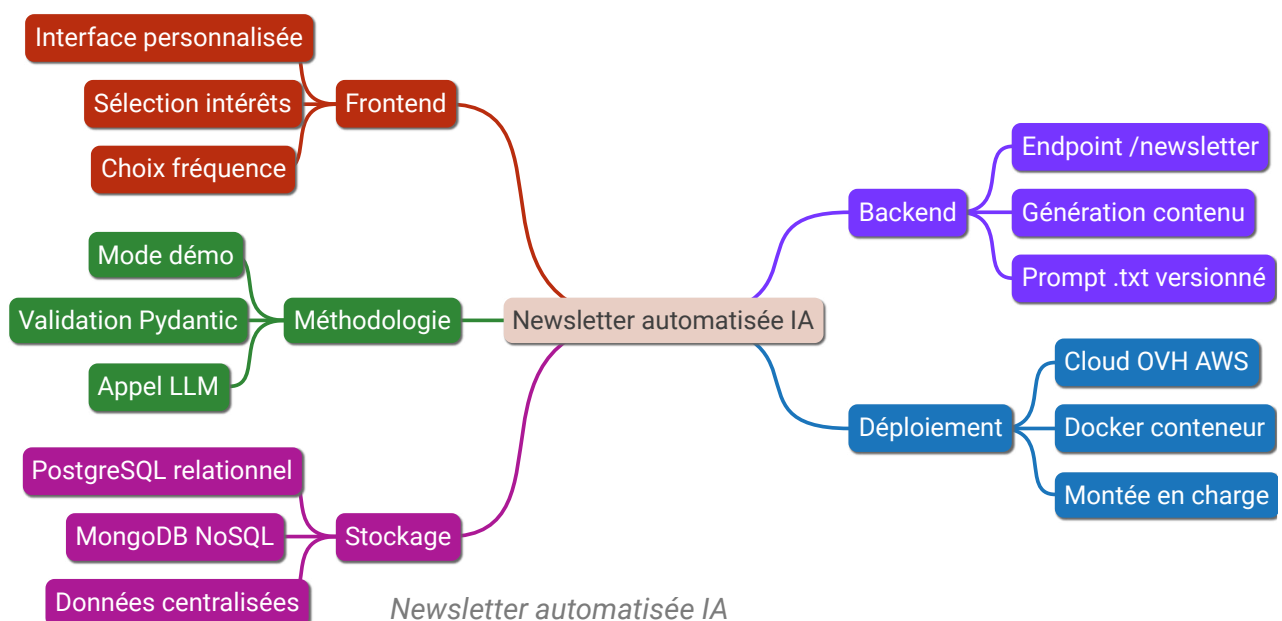
MongoDB

Base de données NoSQL stockant des documents JSON flexibles et évolutifs.

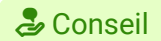
La théorie en Bref

💡 Fondamental

La **newsletter automatisée** grâce à l'**IA** permet de personnaliser l'envoi d'informations selon les **centres d'intérêt** et la **fréquence** choisie par l'utilisateur. Le processus inclut la **création d'une interface frontend** pour la sélection des préférences, la **génération de contenu** via un prompt IA stocké dans un fichier **.txt versionné**, et la mise en place d'un **endpoint backend** pour traiter les requêtes. L'utilisation d'un **mode démo** facilite le test sans appel à l'IA. Enfin, le déploiement sur des services cloud et l'utilisation de bases de données comme **PostgreSQL** ou **MongoDB** permettent la montée en charge et la gestion centralisée des données.



Les conseils pro



Comment personnaliser une newsletter selon les intérêts clients ?

- Proposer une interface claire pour sélectionner intérêts.
- Utiliser des catégories pertinentes et représentatives.
- Permettre la sélection de la fréquence d'envoi.
- Tester la personnalisation avec un mode démo.

Comment intégrer un prompt IA dans un système ?

- Stocker le prompt dans un fichier .txt versionné.
- Charger dynamiquement le prompt dans le backend.
- Paramétrer le prompt avec les données utilisateur.
- Gérer les erreurs et le mode démo.

Comment assurer la montée en charge d'une newsletter IA ?

- Déployer sur un cloud fiable (OVH, AWS).
- Utiliser Docker pour portabilité.
- Centraliser les données dans une base adaptée.
- Optimiser l'hébergement du modèle IA.

Comment garantir la qualité du contenu généré ?

- Définir un prompt clair et structuré.
- Limiter la taille de la réponse (max_tokens).
- Valider la sortie avant envoi.
- Mettre à jour régulièrement le prompt.

IV Auto-évaluation

A. Exercice : Quiz

[solution n°1 p. 29]

Exercice

Vous devez créer un prompt IA pour générer une newsletter automatisée en trois parties (Actualités, Conseils, Promotions) destinée à un jeune actif sensible à l'écologie avec des intérêts spécifiques. Quelle formulation est la plus adaptée ?

Génère une newsletter hebdomadaire en Français, qui comporte 3 sections : actualités, conseils, promotions. Le lecteur est intéressé par : brunchs rapides, bio, café. Rédige des titres accrocheurs et des paragraphes courts.

Crée une newsletter en anglais avec des conseils sur la technologie et les sports.

Écris un article de blog sur les voyages sans mentionner les intérêts du lecteur.

Fais une liste de produits sans personnalisation ni sections distinctes.

Exercice

Pourquoi est-il préférable de stocker le prompt IA dans un fichier .txt versionné plutôt que directement dans le code ?

Pour faciliter la gestion des versions et modifications du prompt sans toucher au code source.

Parce que le code ne peut pas contenir de texte.

Pour éviter que le prompt soit visible par les utilisateurs finaux.

Parce que les fichiers .txt sont plus rapides à exécuter.

Exercice

Dans la création de la newsletter automatisée, quelles fonctionnalités doivent être présentes dans l'interface frontend pour personnaliser la newsletter ?

A Sélection des centres d'intérêt par l'utilisateur

B Choix de la fréquence d'envoi des newsletters

C Saisie du contenu de la newsletter par l'utilisateur

D Saisie de l'adresse email pour l'envoi

Exercice

Quel est le rôle principal de l'endpoint /newsletter dans le backend ?

Recevoir les centres d'intérêt de l'utilisateur et générer la newsletter personnalisée

Envoyer directement les emails aux utilisateurs

Stocker les emails des utilisateurs dans une base de données

Gérer les paiements des abonnements

Exercice

Le mode démo permet de tester la génération de newsletter sans faire appel à l'intelligence artificielle réelle. Vrai ou Faux ?

Vrai

Faux

Exercice

Quels sont les avantages respectifs de PostgreSQL et MongoDB pour stocker les données liées à la newsletter automatisée ?

A

PostgreSQL est adapté aux données structurées avec relations claires entre les données.

B

MongoDB offre une grande souplesse avec des documents JSON non structurés.

C

PostgreSQL est une base de données NoSQL.

D

MongoDB nécessite un schéma rigide et fixe.

Exercice

Pourquoi utiliser des services cloud comme OVH ou AWS combinés à Docker pour déployer la newsletter automatisée ?

A

Pour garantir la portabilité de l'application entre différents serveurs.

B

Pour respecter la souveraineté des données selon le choix du cloud.

C Pour limiter l'accès à un seul utilisateur local.

D Pour augmenter la puissance de calcul et la montée en charge.

Exercice

Quelle structure doit avoir le prompt IA pour générer une newsletter efficace ?

Trois sections distinctes : actualités, conseils, promotions, avec titres accrocheurs et paragraphes courts.

Un seul paragraphe long sans sections.

Une liste de produits sans texte explicatif.

Un texte en anglais sans personnalisation.

Exercice

L'utilisation des classes Pydantic pour les requêtes et réponses permet de valider automatiquement les données échangées avec l'API. Vrai ou Faux ?

Vrai

Faux

Exercice

Dans le composant React NewsletterIA, quelles sont les conditions pour activer le bouton de soumission du formulaire ?

Au moins un centre d'intérêt sélectionné

Un email valide saisi

La fréquence doit être sélectionnée

Le formulaire doit être rempli manuellement

Exercice

Associez les termes aux définitions correspondantes.

A Docker

B MongoDB

C Mode démo

D PostgreSQL

Base de données relationnelle adaptée aux données structurées	Base de données NoSQL stockant des documents JSON flexibles	Fonctionnalité pour tester sans appel réel à l'IA	Outil garantissant la portabilité des applications entre serveurs
---	---	---	---

Exercice

Ordonnez les étapes pour générer une newsletter automatisée avec l'IA.

A Le frontend envoie une requête POST à l'endpoint /newsletter

B Le backend génère la newsletter via le prompt IA

C L'utilisateur sélectionne ses centres d'intérêt et sa fréquence dans le frontend

D Le backend renvoie le texte généré au frontend

E Le frontend affiche la newsletter personnalisée à l'utilisateur

Réponse : ____ _

Exercice

Associez les services cloud à leurs caractéristiques principales.

A AWS

B Localhost

C OVH

D Docker

Respecte la souveraineté des données et normes européennes	Leader mondial du cloud avec grande variété de services	Permet de déployer l'application dans un conteneur portable	Limité à un seul utilisateur sur la même machine
--	---	---	--

Exercice

Le versionnage des fichiers de prompt permet de suivre les modifications et d'améliorer progressivement la qualité des newsletters générées. Vrai ou Faux ?

Vrai

Faux

Exercice

Quelles sont les bonnes pratiques pour personnaliser efficacement une newsletter automatisée ?

- A** Proposer une interface claire pour sélectionner les intérêts
- B** Permettre le choix de la fréquence d'envoi
- C** Utiliser un prompt IA structuré et clair
- D** Demander à l'utilisateur de rédiger le contenu

B. Exercice : Cas pratique

Question

[solution n°2 p. 34]

Vous travaillez pour un magasin local en tant que développeur full stack. Votre but est de générer une newsletter automatisée grâce à l'IA. Pour ce faire, vous devez créer un prompt IA clair et ciblé que vous pourrez ensuite intégrer dans le système. Ce prompt devra permettre à l'IA de générer automatiquement une newsletter en trois parties « *Actualités - conseils - promotions* », pour le profil suivant : « *jeune actif sensible à l'écologie* » et dont les intérêts sont les suivants : « *brunchs rapides, bio, café* ».

Celui-ci devra également être écrit en français et rédigé dans un fichier.txt versionnable.

Livrable attendu : prompt de 3 lignes avec fichier de versionning.

Enfin, expliquez pourquoi il est important de stocker le prompt dans un fichier .txt versionné plutôt que dans le code directement.

Solutions des exercices

Solution n°1

[exercice p. 24]

Exercice

Vous devez créer un prompt IA pour générer une newsletter automatisée en trois parties (Actualités, Conseils, Promotions) destinée à un jeune actif sensible à l'écologie avec des intérêts spécifiques. Quelle formulation est la plus adaptée ?

✓ Génère une newsletter hebdomadaire en Français, qui comporte 3 sections : actualités, conseils, promotions. Le lecteur est intéressé par : brunchs rapides, bio, café. Rédige des titres accrocheurs et des paragraphes courts.

Crée une newsletter en anglais avec des conseils sur la technologie et les sports.

Écris un article de blog sur les voyages sans mentionner les intérêts du lecteur.

Fais une liste de produits sans personnalisation ni sections distinctes.

🔍 Le prompt doit être clair, structuré en trois sections et personnalisé selon les intérêts du lecteur, comme dans la première proposition. Les autres options ne respectent pas ces critères essentiels.

Exercice

Pourquoi est-il préférable de stocker le prompt IA dans un fichier .txt versionné plutôt que directement dans le code ?

✓ Pour faciliter la gestion des versions et modifications du prompt sans toucher au code source.

Parce que le code ne peut pas contenir de texte.

Pour éviter que le prompt soit visible par les utilisateurs finaux.

Parce que les fichiers .txt sont plus rapides à exécuter.

🔍 Stocker le prompt dans un fichier .txt versionné permet de suivre précisément les évolutions, de modifier le prompt sans déployer le code, et de séparer clairement la logique métier du contenu textuel.

Exercice

Dans la création de la newsletter automatisée, quelles fonctionnalités doivent être présentes dans l'interface frontend pour personnaliser la newsletter ?

- ☒ A Sélection des centres d'intérêt par l'utilisateur
- ☒ B Choix de la fréquence d'envoi des newsletters
- ☐ C Saisie du contenu de la newsletter par l'utilisateur
- ☐ D Saisie de l'adresse email pour l'envoi

🔍 L'interface doit permettre à l'utilisateur de choisir ses centres d'intérêt, la fréquence d'envoi, et saisir son email. La saisie du contenu est générée automatiquement par l'IA, donc non demandée à l'utilisateur.

Exercice

Quel est le rôle principal de l'endpoint /newsletter dans le backend ?

☒ Recevoir les centres d'intérêt de l'utilisateur et générer la newsletter personnalisée

☐ Envoyer directement les emails aux utilisateurs

☐ Stocker les emails des utilisateurs dans une base de données

☐ Gérer les paiements des abonnements

🔍 L'endpoint /newsletter reçoit les centres d'intérêt via une requête POST, génère la newsletter personnalisée via l'IA, et renvoie le contenu. Il ne gère pas l'envoi d'emails ni le stockage des données.

Exercice

Le mode démo permet de tester la génération de newsletter sans faire appel à l'intelligence artificielle réelle. Vrai ou Faux ?

☒ Vrai

☐ Faux

🔍 Le mode démo renvoie un texte fictif pour faciliter les tests sans utiliser le modèle IA, ce qui évite les coûts et les temps d'attente.

Exercice

Quels sont les avantages respectifs de PostgreSQL et MongoDB pour stocker les données liées à la newsletter automatisée ?

A PostgreSQL est adapté aux données structurées avec relations claires entre les données.

B MongoDB offre une grande souplesse avec des documents JSON non structurés.

C PostgreSQL est une base de données NoSQL.

D MongoDB nécessite un schéma rigide et fixe.

🔍 PostgreSQL est une base relationnelle idéale pour des données structurées et liées, tandis que MongoDB est une base NoSQL flexible qui stocke des documents JSON avec des structures variables.

Exercice

Pourquoi utiliser des services cloud comme OVH ou AWS combinés à Docker pour déployer la newsletter automatisée ?

A Pour garantir la portabilité de l'application entre différents serveurs.

B Pour respecter la souveraineté des données selon le choix du cloud.

C Pour limiter l'accès à un seul utilisateur local.

D Pour augmenter la puissance de calcul et la montée en charge.

🔍 Le cloud permet la montée en charge et la flexibilité, Docker assure la portabilité, et le choix du cloud peut garantir la conformité réglementaire. Limiter l'accès à un seul utilisateur local n'est pas un objectif.

Exercice

Quelle structure doit avoir le prompt IA pour générer une newsletter efficace ?

✓ Trois sections distinctes : actualités, conseils, promotions, avec titres accrocheurs et paragraphes courts.

Un seul paragraphe long sans sections.

Une liste de produits sans texte explicatif.

Un texte en anglais sans personnalisation.



Un prompt structuré en trois sections claires avec titres et paragraphes courts facilite la génération d'une newsletter lisible et engageante.

Exercice

L'utilisation des classes Pydantic pour les requêtes et réponses permet de valider automatiquement les données échangées avec l'API. Vrai ou Faux ?

✓ Vrai

Faux



Pydantic valide les données selon les modèles définis, assurant la cohérence et la sécurité des échanges avec l'API.

Exercice

Dans le composant React NewsletterIA, quelles sont les conditions pour activer le bouton de soumission du formulaire ?

✓ Au moins un centre d'intérêt sélectionné

Un email valide saisi

La fréquence doit être sélectionnée

Le formulaire doit être rempli manuellement



Le bouton est activé uniquement si l'utilisateur a sélectionné au moins un centre d'intérêt et saisi un email valide. La fréquence a une valeur par défaut et le formulaire n'est pas rempli manuellement pour le contenu.

Exercice

Associez les termes aux définitions correspondantes.

Base de données relationnelle adaptée aux données structurées	Base de données NoSQL stockant des documents JSON flexibles	Fonctionnalité pour tester sans appel réel à l'IA	Outil garantissant la portabilité des applications entre serveurs
PostgreSQL	MongoDB	Mode démo	Docker

🔍 PostgreSQL est relationnel, MongoDB est NoSQL, le mode démo permet de tester sans IA réelle, et Docker assure la portabilité des applications.

Exercice

Ordonnez les étapes pour générer une newsletter automatisée avec l'IA.

A L'utilisateur sélectionne ses centres d'intérêt et sa fréquence dans le frontend

B Le frontend envoie une requête POST à l'endpoint /newsletter

C Le backend génère la newsletter via le prompt IA

D Le backend renvoie le texte généré au frontend

E Le frontend affiche la newsletter personnalisée à l'utilisateur

🔍 La génération suit un flux logique : sélection utilisateur, requête au backend, génération IA, réponse, affichage.

Exercice

Associez les services cloud à leurs caractéristiques principales.

Respecte la souveraineté des données et normes européennes	Leader mondial du cloud avec grande variété de services	Permet de déployer l'application dans un conteneur portable	Limité à un seul utilisateur sur la même machine
OVH	AWS	Docker	Localhost

🔍 OVH est européen et conforme RGPD, AWS est leader mondial, Docker assure portabilité, localhost est limité en accès.

Exercice

Le versionnage des fichiers de prompt permet de suivre les modifications et d'améliorer progressivement la qualité des newsletters générées. Vrai ou Faux ?

✓ Vrai

Faux

🔍 Le versionnage est essentiel pour tracer les évolutions, faciliter les retours en arrière et améliorer la qualité du contenu généré.

Exercice

Quelles sont les bonnes pratiques pour personnaliser efficacement une newsletter automatisée ?

A Proposer une interface claire pour sélectionner les intérêts

B Permettre le choix de la fréquence d'envoi

C Utiliser un prompt IA structuré et clair

D Demander à l'utilisateur de rédiger le contenu



La personnalisation passe par une interface intuitive, choix de fréquence, et un prompt IA bien conçu. L'utilisateur ne rédige pas le contenu.

[exercice p. 28] **Solution n°2**

Tout d'abord, voici le prompt versionné.

« Génère une newsletter hebdomadaire en 3 parties : actualités, conseils, promotions. Le lecteur est : {avatar}. Il s'intéresse principalement à : {interets}.
Rédige des titres accrocheurs et des paragraphes courts. »

Le titre pourrait être : "Prompt_newsletter_v1.0.txt"

Dorénavant, afin de répondre à la question initiale, stocker le prompt dans un fichier .txt versionné plutôt que directement dans le code présente de nombreux avantages.

Premièrement, la séparation du code et du contenu.

En ayant un fichier .txt, le code reste centré sur la logique (à savoir les endpoints, les fonctions, etc.). Ici, les prompts sont traités comme des ressources externes, faciles à modifier sans toucher au code.

Ensuite, grâce à cette méthodologie, c'est plus facile de mettre à jour l'application.

En effet, modifier un prompt dans le fichier .txt ne nécessite pas de redéploier l'application. On peut donc tester rapidement plusieurs variantes de prompts sans risquer de casser le code.

Un autre point important concerne le versionning. Ici, nous trackons chaque prompt avec GIT. Cela permet de savoir qui a changé quoi, quand et pourquoi. Nous pourrions donc facilement revenir à une version précédente si nécessaire.

Enfin, cette approche rend le travail plus collaboratif et évolutif. Dans un projet qui utilise plusieurs prompts, on peut organiser chaque prompts dans un dossier qui comprendrait des fichiers .txt. Chacun serait versionné. Cela permet également de maintenir plusieurs variantes, de comparer leur efficacité et de laisser toute l'équipe en charge du projet (marketeurs, rédacteurs, développeurs, etc.) de contribuer de manière propre, suivie et sécurisée.